```sql
CREATE DATABASE payments;

USE payments;
-- ------------------ CREACION DE LA TABLA transaction -----------
CREATE TABLE IF NOT EXISTS transactions(
    id VARCHAR(255),
    card_id VARCHAR(25),
    business_id VARCHAR(255),
    timestamp VARCHAR(25),
    amount VARCHAR(25),
    declined VARCHAR(25),
    products_id VARCHAR(25),
    user_id VARCHAR(15),
    lat VARCHAR(50),
    longitude VARCHAR(50)
);
```

```
14  10:14:16  CREATE DATABASE payments                                                    1 row(s) affected
15  10:14:24  USE payments                                                               0 row(s) affected
16  10:15:28  CREATE TABLE IF NOT EXISTS transactions( id VARCHAR(255),  card_id VARCHAR(25),  business_id V...  0 row(s) affected
```

✓ Crear DDBB y luego tabla temporal llamada ***transactions***.

```sql
SHOW VARIABLES LIKE 'secure_file_priv';
```

Result Grid | Filter Rows: | Export: | Wrap Cel

Variable_name	Value
secure_file_priv	C:\ProgramData\MySQL\MySQL Server 8.4\Uploads\

✓ Muestra el sitio donde guardar los CSV para que se importen correctamente.

```sql
LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ';'
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
19  10:26:55  LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//transactions.csv' -- CARGAR...  100000 row(s) affected Records: 100000  Deleted: 0  Skipped: 0  Warnings: 0
```

✓ Importar los datos hacia la tabla ***transactions***.

```sql
-- ---------------------- ANALISIS DE DATOS ------------------------------------------------------------------------------
WITH buscar_duplic AS        -- BUSCAR DUPLICADOS
(SELECT *,
ROW_NUMBER() OVER(PARTITION BY id, card_id, business_id, timestamp, amount, declined, products_id, user_id, lat, longitude) AS num_reg
FROM transactions
)
SELECT * FROM buscar_duplic
WHERE num_reg > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	card_id	business_id	timestamp	amount	declined	products_id	user_id	lat	longitude	num_reg

```
23  08:48:24  WITH buscar_duplic AS (SELECT *,  ROW_NUMBER() OVER(PARTITION BY id, card_id, business_id, timest...  0 row(s) returned
```

✓ Luego de cargar los datos en la tabla temporal, se realiza un análisis de los datos para ver si existen duplicados.

```sql
CREATE TABLE IF NOT EXISTS transaction(
    id VARCHAR(255) NOT NULL PRIMARY KEY,
    card_id VARCHAR(25),
    business_id VARCHAR(25),
    timestamp DATETIME,
    amount DECIMAL(10, 2),
    declined TINYINT,
    products_id VARCHAR(25),
    user_id INT,
    lat DECIMAL(10, 6),
    longitude DECIMAL(10, 6)
);
```

```
39  11:52:47  CREATE TABLE IF NOT EXISTS transaction( id VARCHAR(255) NOT NULL PRIMARY KEY,  card_id VAR...  0 row(s) affected
```

```sql
INSERT INTO transaction (id, card_id, business_id, timestamp, amount, declined, products_id, user_id, lat, longitude)
SELECT
    TRIM(id),
    card_id,
    LEFT(business_id, 25),
    STR_TO_DATE(timestamp, '%Y-%m-%d %H:%i:%s'),
    CAST(amount AS DECIMAL(10,2)),
    CAST(declined AS SIGNED),
    products_id,
    CAST(user_id AS SIGNED),
    CAST(lat AS DECIMAL(10,6)),
    CAST(longitude AS DECIMAL(10,6))
FROM transactions;
```

```
40  11:54:46  INSERT INTO transaction (id, card_id, business_id, timestamp, amount, declined, products_id, user_id, lat, lon...  100000 row(s) affected Records: 100000  Duplicates: 0  Warnings: 0
```

```sql
ALTER TABLE transaction                -- MODIFICAR EL NOMBRE DEL CAMPO PARA CONSISTENCIA CON LA TABLA companies
RENAME COLUMN business_id to company_id;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	card_id	company_id	timestamp	amount	declined	products_id	user_id	lat	longitude
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.199929	1.435540

✓ Se modifica el nombre del campo *business_id* a *company_id* para mejorar la consistencia y disminuir e margen de error en futuras consultas.

```sql
DROP TABLE transactions;
```

```
42  12:04:18  DROP TABLE transactions  0 row(s) affected
```

✓ Se crea la tabla final ***transaction*** con y se insertan los datos de los campos verificados procedentes de la tabla flexible ***transactions***, la cual se elimina para solo dejar la tabla final ***transaction***.

```sql
-- ------------------ CREACION DE LA TABLA company -----------

CREATE TABLE IF NOT EXISTS companies(
    company_id VARCHAR(15),
    company_name VARCHAR(50),
    phone VARCHAR(50),
    email VARCHAR(50),
    country VARCHAR(50),
    website VARCHAR(50)
);
```

```
43  12:13:13  CREATE TABLE IF NOT EXISTS companies( company_id VARCHAR(15),  company_name VARCHAR(50), ...  0 row(s) affected
```

```sql
LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//companies.csv'
INTO TABLE companies
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
45  12:16:41  LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//companies.csv' - CARGAR ...  100 row(s) affected Records: 100  Deleted: 0  Skipped: 0  Warnings: 0
```

- ✓ Se crea la tabla de staging companies y se importan los datos desde el CSV, a diferencia del anterior, en este los campos están divididos por comas(,).

```sql
-- ----------------------- ANALISIS DE LOS DATOS ------------------

SELECT company_id, COUNT(*) AS Cant_veces FROM companies
GROUP BY company_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_id	Cant_veces

```
48  12:21:49  SELECT company_id, COUNT(*) AS Cant_veces FROM companies GROUP BY company_id HAVING COUN...  0 row(s) returned
```

```sql
SELECT DISTINCT company_name
FROM companies;
```

Result Grid | Filter Rows:

company_name
Maecenas Malesuada Fringilla Inc.
Magna A Neque Industries

- ✓ Se verifica si hay duplicados en el nombre de las compañías.

```sql
UPDATE companies            -- ELIMINAR EL (.) FINAL DE ALGUNAS COMPAÑIAS
SET company_name = TRIM(TRAILING '.' FROM company_name)
WHERE company_name LIKE '%.' AND company_id IS NOT NULL;
```

```
1  09:27:53  UPDATE companies SET company_name = TRIM(TRAILING '.' FROM company_name) WHERE company_nam...  17 row(s) affected Rows matched: 17 Changed: 17 Warnings: 0
```

- ✓ Se elimina el punto final (.) que tenían algunas compañías para mejorar la consistencia de los datos y facilitar las consultas.

```
111 •   ALTER TABLE companies
112     RENAME COLUMN company_id to id;
113
114 •   SELECT * FROM companies;
115
```

Result Grid | Filter Rows:

id	company_name	phc
b-2222	Ac Fermentum Incorporated	06 8

```
11  10:29:37  SELECT * FROM companies  100 row(s) returned
```

- ✓ Modifico el nombre del campo a **id**, el cual constituye la PK y será más factible a la hora de desarrollar consultas.

```
116 •  CREATE TABLE IF NOT EXISTS company (          -- TABLA FINAL
117        id VARCHAR(25) NOT NULL PRIMARY KEY,
118        company_name VARCHAR(50),
119        phone VARCHAR(25),
120        email VARCHAR(50) UNIQUE,
121        country VARCHAR(50),
122        website VARCHAR(50)
123    );
124
125 •  INSERT INTO company (id, company_name, phone, email, country, website)
126    SELECT id, company_name, phone, email, country, website
127    FROM companies;
```

```
12  10:40:23  CREATE TABLE IF NOT EXISTS company (  -- TABLA FINAL id VARCHAR(25) NOT NULL PRIMARY K...  0 row(s) affected
14  10:45:12  SELECT * FROM company  100 row(s) returned
```

```
129 •  DROP TABLE companies;
```

```
15  10:50:01  DROP TABLE companies  0 row(s) affected
```

- ✓ Se crea la tabla final donde declaro la PK y luego se insertan los datos limpios y modificados desde la tabla staging companies, la cual se elimina para dejar la tabla final **company**.

```sql
-- ------------------ CREACION DE LA TABLA credit_card -----------

CREATE TABLE IF NOT EXISTS credit_cards(
    id VARCHAR(20),
    user_id VARCHAR(50),
    iban VARCHAR(50),
    pan VARCHAR(50),
    pin VARCHAR(4),
    cvv VARCHAR(3),
    track1 VARCHAR(255),
    track2 VARCHAR(255),
    expiring_date VARCHAR(15)
);

LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//credit_cards.csv'
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

✓	21	12:15:46	CREATE TABLE IF NOT EXISTS credit_cards(id VARCHAR(20), user_id VARCHAR(50), iban VARCHA...	0 row(s) affected
✓	22	12:15:59	LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//credit_cards.csv' -- CARGAR...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

```sql
CREATE TABLE IF NOT EXISTS credit_card(
    id VARCHAR(20) NOT NULL PRIMARY KEY,
    user_id INT,
    iban VARCHAR(50),
    pan VARCHAR(35),
    pin CHAR(4),
    cvv CHAR(3),
    expiring_date DATE
);
```

✓	30	18:22:49	SELECT * FROM credit_cards	5000 row(s) returned

```sql
INSERT INTO credit_card (id, user_id, iban, pan, pin, cvv, expiring_date)
SELECT
    id,
    CAST(user_id AS SIGNED),
    iban,
    LEFT (pan, 35),
    CAST(pin AS CHAR(4)),
    CAST(cvv AS CHAR(3)),
    STR_TO_DATE (expiring_date, '%m/%d/%y')
FROM credit_cards;

DROP TABLE credit_cards;
```

- ✓ Se crea la tabla final **credit_card** sin tener en cuenta los campos track1 y track2 ya que no se necesitan, se define la PK y luego se insertan los datos desde la staging table credit_cards, la cual se elimina finalmente.

```sql
-- ------------------ CREACION DE LA TABLA data_user -----------

CREATE TABLE IF NOT EXISTS american_users(            -- TABLA american_users
    id VARCHAR(20),
    name VARCHAR(50),
    surname VARCHAR(50),
    phone VARCHAR(50),
    email VARCHAR(50),
    birth_day VARCHAR(50),
    country VARCHAR(50),
    city VARCHAR(50),
    postal_code VARCHAR(15),
    address VARCHAR(50)
);
```

```
41  21:04:35  CREATE TABLE IF NOT EXISTS american_users( id VARCHAR(20),  name VARCHAR(50),  surname VA...  0 row(s) affected
```

```sql
LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//american_users.csv'
INTO TABLE american_users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
42  21:05:03  LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//american_users.csv' -- CAR...  1010 row(s) affected Records: 1010  Deleted: 0  Skipped: 0  Warnings: 0
```

- ✓ Se crea la staging table *american_users* y se importan los datos desde el CSV.

```sql
-- ---------------------- ANALISIS DE LOS DATOS -----------------

UPDATE american_users
SET phone = CONCAT(
  '+1',
  REPLACE(
    REPLACE(
      REPLACE(
        REPLACE(
          REPLACE(phone, '+1', ''),
        ' ', ''),
      '-', ''),
    '(', ''),
  ')', '')
)
WHERE phone IS NOT NULL;

UPDATE american_users
SET phone = REPLACE(phone, '+11', '+1');

ALTER TABLE american_users              -- RENOMBRAR LA COLUMNA
RENAME COLUMN birth_day TO birth_date;

UPDATE american_users                                   -- CONVIERTO COLUMNA A DATE
SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y');
```

- ✓ Se limpian los datos tratando de estandarizar el campo *phone,* en el cual se deja intencionalmente algunos números con **+1+** ya que en la vida real les faltaría un número. Además, se renombra el campo birth_date (creado por error como birth_day) y se modifica el formato de los datos del mismo a DATE.

```sql
CREATE TABLE IF NOT EXISTS european_users(              -- TABLA european_users
    id VARCHAR(20),
    name VARCHAR(50),
    surname VARCHAR(50),
    phone VARCHAR(50),
    email VARCHAR(50),
    birth_date VARCHAR(50),
    country VARCHAR(50),
    city VARCHAR(50),
    postal_code VARCHAR(15),
    address VARCHAR(50)
);

LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//european_users.csv' -- CARGAR ARCHIVO CSV
INTO TABLE european_users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
60  10:55:41  CREATE TABLE IF NOT EXISTS european_users(        -- TABLA european_users id VARCHAR(20),   na...  0 row(s) affected
61  10:55:48  LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//european_users.csv' -- CAR...  3990 row(s) affected Records: 3990  Deleted: 0  Skipped: 0  Warnings: 0
```

- ✓ Se crea la staging table *european_users* y se importan los datos desde el CSV.

```
253     -- ---------------------- ANALISIS DE LOS DATOS -----------------
254
255     UPDATE european_users                                        -- CONVIERTO COLUMNA A DATE
256     SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y');
257
258     UPDATE european_users                                -- Estandarizar campo phone
259     SET phone = REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(phone, '+', ''),' ', ''),'-', ''),'(', ''),')', '');
```

```
261     CREATE TABLE IF NOT EXISTS data_user(                -- TABLA FINAL
262         id INT NOT NULL PRIMARY KEY,
263         name VARCHAR(50),
264         surname VARCHAR(50),
265         phone VARCHAR(50),
266         email VARCHAR(50),
267         birth_date DATE,
268         country VARCHAR(50),
269         city VARCHAR(50),
270         postal_code VARCHAR(15),
271         address VARCHAR(50)
272     );
273
274     INSERT INTO data_user (id, name, surname, phone, email, birth_date, country, city, postal_code, address)
275     SELECT CAST(id AS SIGNED), name, surname, phone, email, birth_date, country, city, postal_code, address
276     FROM european_users
277     UNION ALL
278     SELECT CAST(id AS SIGNED), name, surname, phone, email, birth_date, country, city, postal_code, address
279     FROM american_users;
```

```
281     DROP TABLE american_users, european_users;
```

```
85  12:38:49  CREATE TABLE IF NOT EXISTS data_user(        -- TABLA FINAL id INT NOT NULL PRIMARY KEY,   n...  0 row(s) affected
86  12:38:55  INSERT INTO data_user (id, name, surname, phone, email, birth_date, country, city, postal_code, address) SE...  5000 row(s) affected Records: 5000  Duplicates: 0  Warnings: 0
87  12:48:41  DROP TABLE american_users, european_users                                                                      0 row(s) affected
```

- ✓ Se crea la tabla final ***data_user***, en la cual insertamos todos los datos provenientes de las staging tables *american_users* y *european_users,* al tener ambas los mismos campos se utiliza UNION ALL para unir los registros. Al final, se eliminan las dos tablas flexibles.

```sql
-- --------- CREAR LAS FOREIGN KEYS ----------------------------------------------------

ALTER TABLE transaction
ADD CONSTRAINT fk_company
FOREIGN KEY(company_id) REFERENCES company(id),
ADD CONSTRAINT fk_data_user
FOREIGN KEY(user_id) REFERENCES data_user(id),
ADD CONSTRAINT fk_credit_card
FOREIGN KEY(card_id) REFERENCES credit_card(id);
```

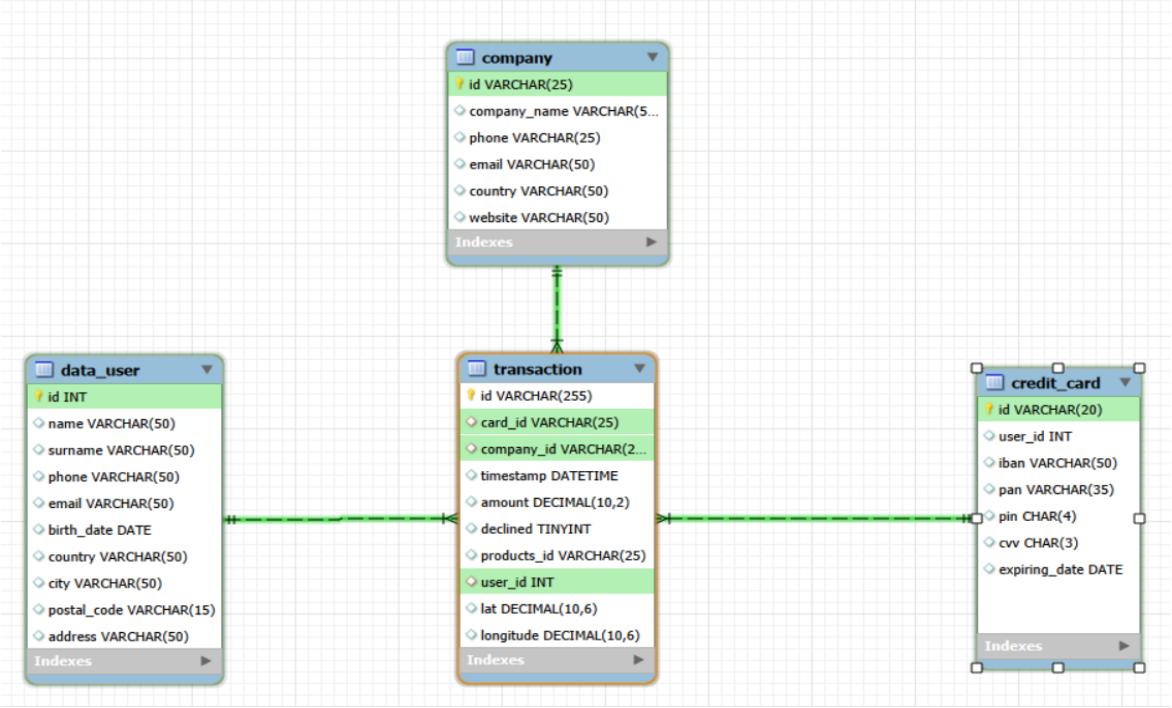
```
89  12:57:53  ALTER TABLE transaction ADD CONSTRAINT fk_company FOREIGN KEY(company_id) REFERENCES co...  100000 row(s) affected Records: 100000  Duplicates: 0  Warnings: 0
```

```sql
SELECT                  -- SOLO PARA SABER LAS FK DE UNA TABLA ESPECIFICA.
    COLUMN_NAME,
    CONSTRAINT_NAME,
    REFERENCED_TABLE_NAME,
    REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE
    TABLE_NAME = 'transaction'
    AND TABLE_SCHEMA = 'payments'
    AND REFERENCED_TABLE_NAME IS NOT NULL;
```

COLUMN_NAME	CONSTRAINT_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
company_id	fk_company	company	id
card_id	fk_credit_card	credit_card	id
user_id	fk_data_user	data_user	id

```
90  13:04:40  SELECT  -- SOLO PARA SABER LAS FK DE UNA TABLA ESPECIFICA.  COLUMN_NAME,  CO...  3 row(s) returned
```

- ✓ Se crean las llaves secundarias en la tabla *transaction*, la cual se asume como tabla de **Hechos** y luego se verifica que hayan sido creadas correctamente.



Nivel 1

```sql
-- ---------------------------- NIVEL 1 ---------------------------------------------

-- 1-) Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones

SELECT d.*
FROM data_user d
JOIN (SELECT user_id, COUNT(*) AS Cant_trasacc
      FROM transaction
      WHERE declined =0
      GROUP BY user_id
      HAVING COUNT(id) > 80) AS t
ON d.id = t.user_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name	surname	phone	email	birth_date	country	city	postal_code	address
185	Molly	Gilliam	08001208023	donec@outlook.couk	1993-12-21	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.
289	Dxwgi	Hwcru	983098797	dxwgi.hwcru@example.com	1976-08-20	Germany	Stuttgart	70173	82 Hwcru Street
318	Bnyr	Astuw	331209644	bnyr.astuw@example.com	1974-05-03	Italy	Genoa	16100	53 Astuw Street

107 19:19:41 SELECT d.* FROM data_user d JOIN (SELECT user_id, COUNT(*) AS Cant_trasacc FROM transaction W... 3 row(s) returned

- ✓ La subconsulta se crea para obtener el id de los clientes que cumplan la condición solicitada de más de 80 transacciones, se asumen solo las aprobadas. Luego mediante el JOIN se unen las tablas y se muestran los datos de los clientes que cumplen con las mismas.

```sql
-- 2-) Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd.

SELECT cc.iban, ROUND(AVG(t.amount), 2) AS Average FROM credit_card cc
JOIN transaction t ON cc.id = t.card_id
JOIN company c ON t.company_id = c.id
WHERE c.company_name = 'Donec Ltd' AND t.declined = 0
GROUP BY cc.iban;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	Average
XX9114064011255863075868O5	356.25
SK9446370242474562577506	142.96
XX7767529178459529755556 40	257.37
XX4138273622897193049089 90	139.59
XX3477872460707696107803 08	240.41

120 19:46:27 SELECT cc.iban, ROUND(AVG(t.amount), 2) AS Average FROM credit_card cc JOIN transaction t ON cc.id = ... 370 row(s) returned

- ✓ Se utiliza JOIN para unir las 3 tablas donde se encuentra la información solicitad y se tienen en cuenta solo las transacciones aprobadas.

Nivel 2

```
326     -- ---------------------------- NIVEL 2 ---------------------------------------------

338 ●   CREATE TABLE IF NOT EXISTS card_status(
339         id INT AUTO_INCREMENT PRIMARY KEY,
340         card_id VARCHAR(20),
341         status VARCHAR(50) CHECK(status IN('active', 'inactive')),
342         CONSTRAINT fk_credit_card2
343         FOREIGN KEY(card_id) REFERENCES credit_card(id)
344     );
```

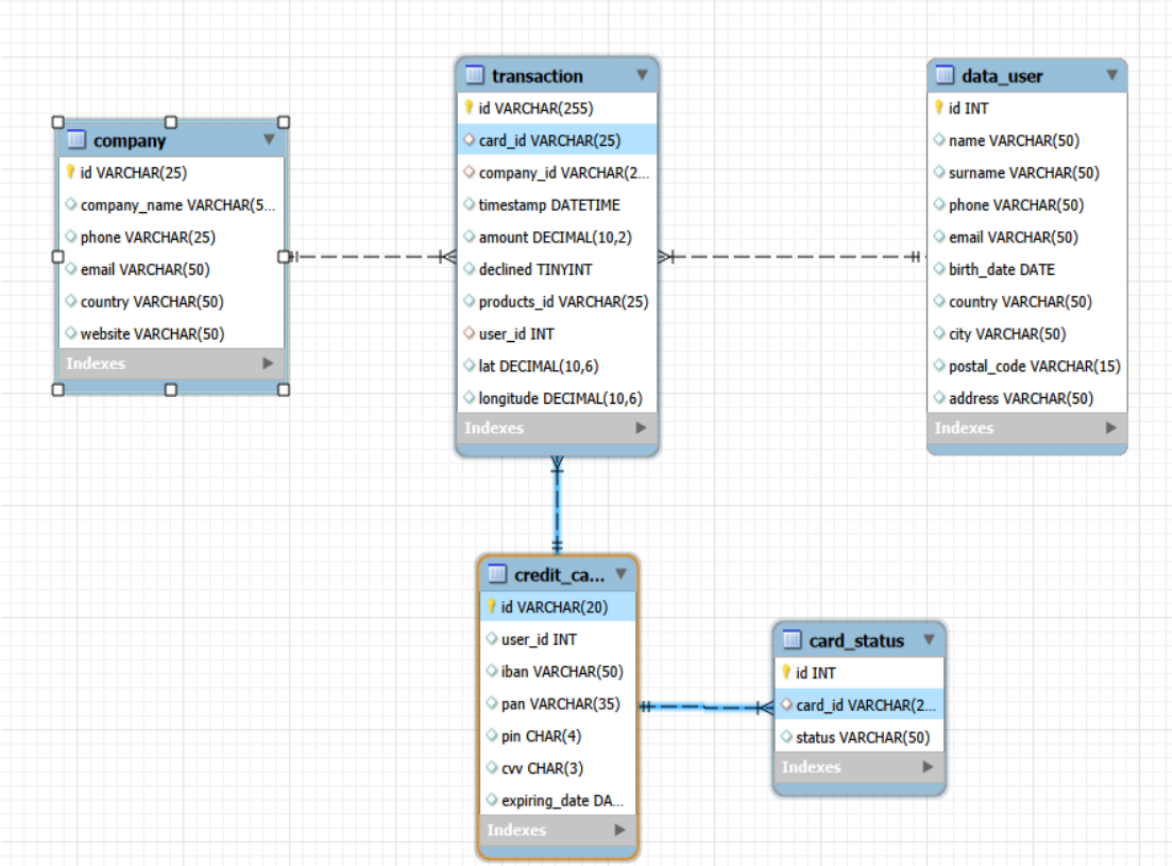
141 11:32:46 CREATE TABLE IF NOT EXISTS card_status(id INT AUTO_INCREMENT PRIMARY KEY, card_id VARC... 0 row(s) affected

✓ Se crea la nueva tabla, la cual tendrá 3 campos: *id* su identificador , *card_id* la foreign key que la relaciona con la tabla padre credit_card y *status* que mostrará el estado actual de la tarjeta con la constraint CHECK, pudiendo ser activa o inactiva.

```
346 ●   INSERT INTO card_status (card_id, status)
347     SELECT card_id,
348           CASE
349               WHEN MIN(declined) = 0 THEN 'active'
350               ELSE 'inactive'
351           END AS status
352     FROM (
353         SELECT card_id, declined,
354               ROW_NUMBER() OVER(PARTITION BY card_id ORDER BY timestamp DESC) AS ranking
355         FROM transaction) AS t
356     WHERE ranking <= 3
357     GROUP BY card_id;
```

171 21:11:19 INSERT INTO card_status (card_id, status) SELECT card_id, CASE WHEN MIN(declined) = 0 TH... 5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

✓ Para insertar los datos se utiliza una subconsulta con la función de ventana ROW_NUMBER mediante la cual se obtienen las ultimas transacciones por tarjeta, se filtran las últimas tres con WHERE y se utiliza CASE para evaluar la condición de que si al menos hay una cero, pues la tarjeta sea activa, sino se asume como inactiva.




```
373     -- 1-) ¿Cuántas tarjetas están activas?
374
375 ●   SELECT COUNT(*) AS Num_active FROM card_status
376     WHERE status = 'active';
```

Result Grid | Filter Rows: | Export:

Num_active
4995

172 21:45:36 SELECT COUNT(*) AS Num_active FROM card_status WHERE status = 'active' 1 row(s) returned

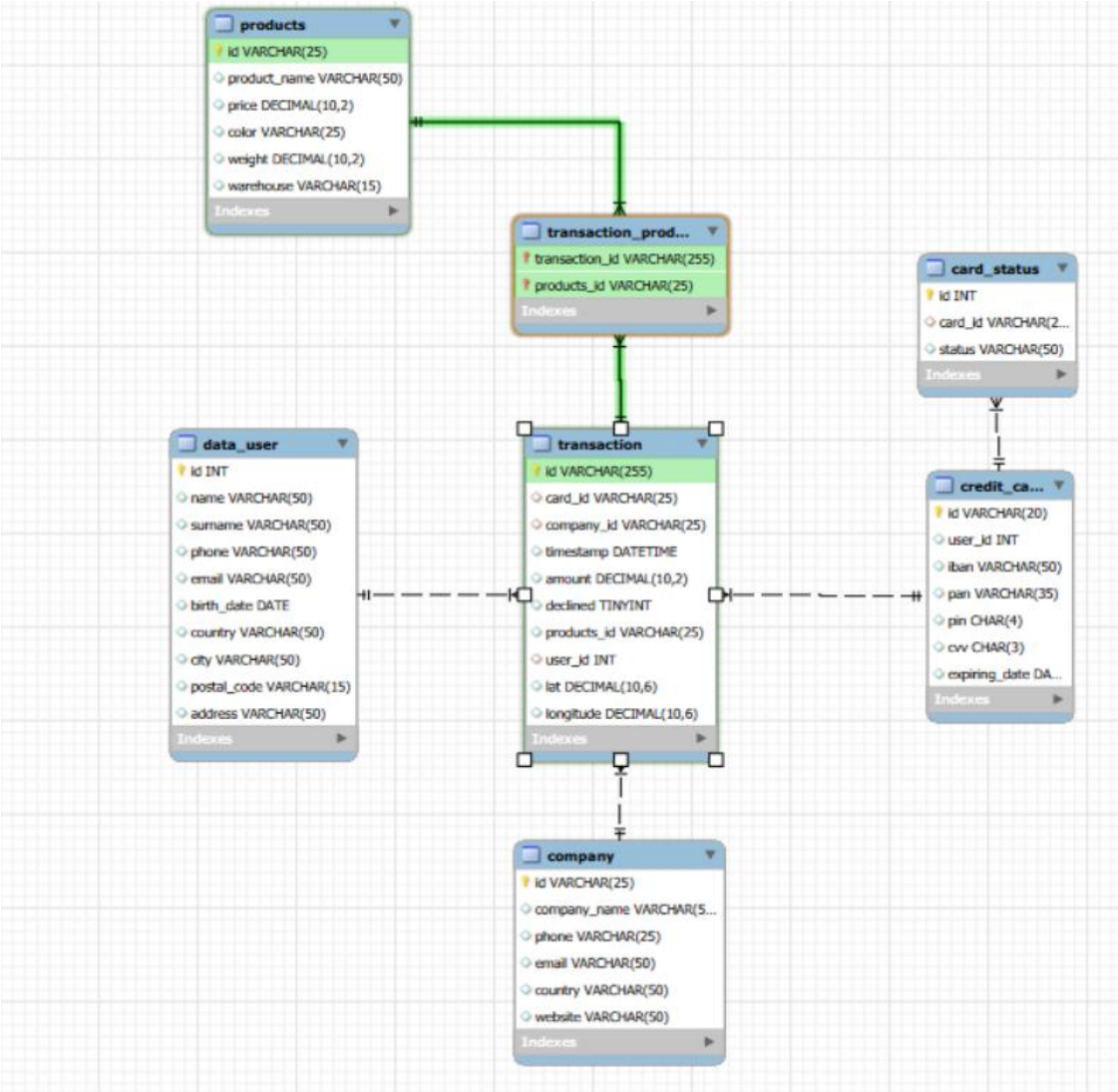

```sql
-- ---------------------------- NIVEL 3 ----------------------------------------

CREATE TABLE IF NOT EXISTS products(
    id VARCHAR(25) NOT NULL PRIMARY KEY,
    product_name VARCHAR(50),
    price DECIMAL(10,2),
    color VARCHAR(25),
    weight DECIMAL(10,2),
    warehouse VARCHAR(15)
);

LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//products.csv' -
INTO TABLE products
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id, product_name, @price, color, @weight, warehouse)                              -
SET
price = CAST(REPLACE(@price, '$', '') AS DECIMAL(10, 2)),
weight = CAST(@weight AS DECIMAL(10, 2));
```

```
173  09:25:58  CREATE TABLE IF NOT EXISTS products( id VARCHAR(25) NOT NULL PRIMARY KEY,   product_name V...  0 row(s) affected
183  10:38:59  LOAD DATA INFILE 'C://ProgramData//MySQL//MySQL Server 8.4//Uploads//products.csv' -- CARGAR A...  100 row(s) affected Records: 100  Deleted: 0  Skipped: 0  Warnings: 0
```

```sql
CREATE TABLE IF NOT EXISTS transaction_product(
    transaction_id VARCHAR(255),
    products_id VARCHAR(25),
    PRIMARY KEY(transaction_id, products_id),
    CONSTRAINT fk_trasaction
        FOREIGN KEY(transaction_id) REFERENCES transaction(id),
    CONSTRAINT fk_products
        FOREIGN KEY(products_id) REFERENCES products(id)
);
```

```
186  11:16:23  CREATE TABLE IF NOT EXISTS transaction_product( transaction_id VARCHAR(255),   products_id VARCH...  0 row(s) affected
```

- ✓ Se crea una tabla intermedia para separar las filas debido a que la columna que tiene la tabla transaction es multivaluada que constituye un diseño no normalizado y así poder gestionar la relación **N:M**.




```
398 •    INSERT INTO transaction_product (transaction_id, products_id)
399          SELECT t.id, p.id FROM transaction t
400          JOIN products p ON FIND_IN_SET (p.id, REPLACE (t.products_id, ' ', ''));
```

241 11:34:18 INSERT INTO transaction_product (transaction_id, products_id) SELECT t.id, p.id FROM transaction t JOIN pr... 253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0

- ✓ Se completan los registros de la tabla intermedia utilizando la función FIND_IN_SET para buscar el valor del id de producto dentro de la lista de elementos separados por comas en

la FK (products_id) de la tabla *transaction*, además se eliminan los espacios mediante REPLACE para que no me falsee la información la función utilizada.

```
402     -- 1-) Necesitamos conocer el número de veces que se ha vendido cada producto.
403
404 ●   SELECT p.id, p.product_name, COUNT(tp.products_id) AS Num_times FROM products p
405     JOIN transaction_product tp ON p.id = tp.products_id
406     JOIN transaction t ON tp.transaction_id = t.id
407     WHERE t.declined = 0
408     GROUP BY p.id, p.product_name
409     ORDER BY Num_times DESC;
410
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	product_name	Num_times
52	riverlands the duel	2642
29	Tully maester Tarly	2627
21	duel Direwolf	2603
16	the duel warden	2602
33	duel warden	2593

244 11:41:08 SELECT p.id, p.product_name, COUNT(tp.products_id) AS Num_times FROM products p JOIN transaction_pr... 100 row(s) returned

✓ Para calcular el número de veces que se ha vendido cada producto, se utiliza la tabla intermedia para resolver la relación N:M y filtro por transacciones no declinadas para tener las ventas reales, agrupando por el identificador y el nombre del producto.